

Guida completa al progetto di navigazione Dublino

Nodi, way, collegamenti e direzioni

1. Scopo del progetto

Il progetto trasforma dati stradali OpenStreetMap in un problema di pianificazione. Il sistema:

1. acquisisce una rete stradale di Dublino; 2. legge nodi, strade, direzioni, velocità e semafori; 3. costruisce un grafo semplificato degli incroci; 4. sceglie automaticamente oppure riceve dall'utente partenza e arrivo; 5. genera un problema PDDL+; 6. usa il planner ENHSP per trovare un percorso; 7. mostra percorso e statistiche nella webapp; 8. può convertire la rete per una visualizzazione separata in SUMO.

Flusso generale:

```
OpenStreetMap
  |
  v
file .osm
  |
  v
grafo stradale contratto
  |
  v
selezione START e GOAL
  |
  v
domain.pddl + problem.pddl
  |
  v
planner ENHSP
  |
  v
piano: start-move nodoA nodoB
  |
  v
percorso e statistiche nella webapp
```

2. Struttura delle cartelle

La cartella principale **files** contiene:

```
files/
|-- build_problems.py
|-- convert_to_osm.py
|-- download_dublin_map.py
|-- sumo_visualize.py
|-- dublin_streets.graphml
|-- dublin_map.png
|-- cache/
|-- cfg_files/
|-- encoder/
|-- net_files/
|-- osm_files/
|-- pddl_files/
```

```
`-- webapp/
```

Ruolo delle cartelle:

- **osm_files**: reti stradali originali in formato OpenStreetMap XML.
- **net_files**: reti convertite nel formato XML di SUMO.
- **pddl_files**: dominio, problemi PDDL e risultati del planner.
- **cfg_files**: configurazioni e percorsi per SUMO.
- **encoder**: script e report leggibili sulle strade OSM.
- **webapp**: applicazione Flask e interfaccia grafica.
- **cache**: risposte o dati temporanei prodotti dagli strumenti geografici.

3. File OSM: nodi e strade

Un nodo OSM è un punto geografico:

```
<node id="12639664" lat="53.3449816" lon="-6.2720865"/>
```

Una way contiene la sequenza dei nodi che formano un tratto:

```
<way changeset="1" id="3716925">
  <nd ref="12639664"/>
  <nd ref="262366032"/>
  <tag k="highway" v="tertiary"/>
  <tag k="name" v="O'Donovan Rossa Bridge"/>
  <tag k="maxspeed" v="50"/>
  <tag k="oneway" v="yes"/>
</way>
```

La relazione fondamentale è:

```
<nd ref="12639664"> ---> <node id="12639664">
```

Il programma unisce le coordinate dei nodi nello stesso ordine degli **<nd>**.

Con **oneway=yes** il collegamento è diretto soltanto dal primo nodo verso il successivo. Senza senso unico viene aggiunto anche il collegamento inverso.

4. Acquisizione delle mappe

download_dublin_map.py usa OSMnx per scaricare tre zone:

- **piccola_centro**: centro di Dublino, raggio 400 metri;
- **media_residenziale**: Ranelagh/Donnybrook, raggio 1200 metri;
- **grande_porto**: Docklands/Port, raggio 3000 metri.

La chiamata principale è:

```
G = ox.graph_from_point(
    coordinate,
    dist=raggio,
    network_type="drive",
    simplify=False,
)
```

`network_type="drive"` richiede una rete percorribile in automobile. `simplify=False` mantiene i nodi intermedi. Il grafo viene poi salvato in XML OSM con `ox.save_graph_xml`.

5. Lettura e filtraggio delle strade

La funzione `build_contracted_graph` si trova sia in `build_problems.py` sia in `webapp/app.py`.

Prima legge tutti i nodi e conserva:

- ID OSM;
- latitudine;
- longitudine;
- eventuale nome;
- presenza del tag `highway=traffic_signals`.

Poi legge le way e conserva solo tipi stradali percorribili, come:

`motorway, trunk, primary, secondary, tertiary, residential, unclassified, living_street` e relativi link

Per ogni way estrae:

- lista ordinata degli ID dei nodi;
- velocità massima, con valore predefinito 30 km/h;
- senso unico.

La velocità viene convertita da km/h a m/s:

$$\text{m/s} = \text{km/h} * 1000 / 3600$$

6. Individuazione degli incroci

Il progetto considera incrocio un nodo che:

- è il primo nodo di una way;
- è l'ultimo nodo di una way;
- compare in almeno due way.

I nodi intermedi che descrivono soltanto la forma della strada non diventano necessariamente nodi del problema PDDL.

Esempio originale:

A -> x1 -> x2 -> B

Se `x1` e `x2` non sono incroci, il grafo contratto diventa:

A -----> B

La distanza dell'arco A -> B è la somma delle distanze:

A-x1 + x1-x2 + x2-B

7. Calcolo delle distanze

Il progetto usa la formula Haversine per calcolare la distanza in metri tra due coordinate geografiche.

Questa distanza viene calcolata tra ogni coppia consecutiva di nodi della way e accumulata fino al successivo incrocio.

8. Costruzione del grafo diretto

Il grafo è rappresentato come una lista di adiacenza:

```
adj[nodo_partenza][nodo_arrivo] = (distanza, velocita)
```

Per una strada a senso unico viene creato:

```
A -> B
```

Per una strada a doppio senso vengono creati:

```
A -> B  
B -> A
```

Se esistono più archi tra gli stessi nodi, il programma conserva quello con la distanza minore.

9. Selezione del sottografo

Le reti reali possono contenere molti nodi. La funzione `select_connected_subgraph` ne seleziona un numero massimo.

Procedura:

1. sceglie come seme il nodo con più collegamenti uscenti;
2. inserisce i suoi vicini nella frontiera;
3. calcola il centro geografico dei nodi già selezionati;
4. prende dalla frontiera il nodo più lontano dal centro;
5. aggiunge i suoi vicini;
6. continua fino al numero massimo richiesto.

In questo modo viene mantenuta la connessione, cercando contemporaneamente di distribuire i nodi sul territorio.

10. Come viene scelto automaticamente lo START

In `build_problems.py` e `webapp/app.py` viene calcolato il numero di archi uscenti di ogni nodo:

```
out_deg = Counter(a for a, _ in edges)  
start = max(selected, key=lambda n: out_deg[n])
```

Lo START automatico è quindi il nodo selezionato con il maggior numero di possibili uscite.

Non viene scelto dal file OSM e non è necessariamente il primo nodo presente nel file.

11. Controllo dei nodi raggiungibili

Dallo START viene eseguita una visita BFS sul grafo diretto:

```
reach = {start}  
queue = deque([start])  
  
while queue:  
    current = queue.popleft()  
    for (a, b) in edges:  
        if a == current and b not in reach:
```

```
reach.add(b)
queue.append(b)
```

La BFS segue soltanto archi consentiti. Di conseguenza rispetta i sensi unici. L'insieme **reach** contiene tutti i nodi ai quali e possibile arrivare.

12. Come viene scelto automaticamente il GOAL

Il GOAL viene scelto soltanto tra i nodi raggiungibili dallo START.

Il programma calcola la distanza geografica tra START e ogni candidato e sceglie quello piu lontano:

```
goal = max(
    reach - {start},
    key=lambda n: haversine(start_lat, start_lon, n_lat, n_lon),
)
```

Questo non e ancora il calcolo del percorso. Serve soltanto a scegliere una destinazione valida e abbastanza distante.

13. Scelta manuale nella webapp

Dopo il caricamento del file OSM, **/api/generate** restituisce:

- nodi selezionati;
- archi diretti;
- START automatico;
- GOAL automatico;
- coordinate e semafori.

La pagina mostra i nodi su Leaflet. Cliccando un nodo l'utente puo impostarlo come START o GOAL.

Quando cambia lo START, JavaScript esegue nuovamente una BFS con **computeReachable**. I nodi non raggiungibili vengono disabilitati come GOAL.

Se il GOAL precedente non e piu raggiungibile, la pagina prova prima il GOAL automatico e poi cerca il nodo raggiungibile piu lontano.

14. Controlli del backend

Premendo il pulsante di risoluzione, il browser invia a **/api/solve**:

```
{
  "token": "identificatore-sessione",
  "start": "nodo_pddl_partenza",
  "goal": "nodo_pddl_arrivo"
}
```

Il backend controlla che:

- la sessione esista;
- START e GOAL corrispondano a nodi conosciuti;
- siano diversi;
- il GOAL sia raggiungibile dallo START rispettando i sensi unici.

Solo dopo questi controlli viene generato il problema PDDL.

15. Conversione dei nomi per PDDL

Gli ID e i nomi OSM non sono sempre validi come identificatori PDDL. `name_map_for` crea nomi senza spazi o caratteri speciali.

Se un nodo ha un nome OSM, viene applicata `slugify`. In caso contrario viene usata una forma simile a:

```
n1234567
```

derivata dalle ultime cifre dell'ID OSM.

Il backend conserva sia la mappa OSM -> PDDL sia quella inversa PDDL -> OSM.

16. Generazione del problema PDDL

Lo stato iniziale contiene la partenza:

```
(:init
  (at nodo_start)
  (= (total-dist) 0)
  (= (total-time) 0)
)
```

Ogni arco del grafo diventa un fatto:

```
(road nodoA nodoB)
```

e possiede valori numerici:

```
(= (distance nodoA nodoB) 120)
(= (speed nodoA nodoB) 8.33)
(= (progress nodoA nodoB) 0)
```

Per ogni nodo viene aggiunto il ritardo semaforico:

```
(= (signal-delay nodoA) 30)
```

oppure zero quando il nodo non è un semaforo.

Il goal è:

```
(:goal (at nodo_goal))
```

La metrica richiesta è:

```
(:metric minimize (total-time))
```

17. Funzionamento di `domain.pddl`

Il dominio definisce tre concetti principali:

```
(at ?l)
(road ?from ?to)
(moving ?from ?to)
```

Azione `start-move`

Può iniziare se il veicolo è nel nodo di partenza e l'arco esiste:

```
(at ?from)
```

```
(road ?from ?to)
```

L'azione rimuove **at ?from** e attiva **moving ?from ?to**.

Processo **driving**

Mentre il veicolo si muove, aumenta continuamente **progress** in funzione della velocità dell'arco.

Evento **arrive**

Quando **progress** raggiunge la distanza:

- termina il movimento;
- inserisce **(at ?to)**;
- somma la distanza totale;
- somma il tempo **distance/speed**;
- somma il ritardo del semaforo di arrivo.

18. Ruolo del planner ENHSP

ENHSP riceve:

```
domain.pddl + problem.pddl
```

Il planner cerca una sequenza di azioni che trasformi:

```
(at START)
```

in:

```
(at GOAL)
```

minimizzando **total-time**.

Il percorso può apparire nel piano come:

```
0.000: (start-move nodoA nodoB)
12.500: (start-move nodoB nodoC)
25.000: (start-move nodoC nodoD)
```

La funzione **parse_plan** legge queste azioni e ricostruisce:

```
nodoA -> nodoB -> nodoC -> nodoD
```

Quindi ENHSP sceglie il percorso PDDL; START e GOAL sono già stati scelti prima del suo avvio.

19. Statistiche del percorso

Dopo la soluzione, il backend ripercorre gli archi trovati e calcola:

- distanza totale;
- tempo di guida;
- numero di semafori attraversati;
- ritardo totale dei semafori;
- tempo impiegato da ENHSP per pianificare.

Il tempo di viaggio e:

```
somma(distanza_arco / velocita_arco) + ritardi_semaforici
```

20. Visualizzazione nella webapp

`webapp/app.py` e il backend Flask. `webapp/templates/index.html` contiene l'interfaccia e il codice JavaScript.

La mappa usa Leaflet e mostra:

- archi della rete;
- nodi disponibili;
- START in verde;
- GOAL in rosso;
- semafori;
- percorso restituito dal planner.

Il percorso e disegnato unendo le coordinate dei nodi nell'ordine restituito da ENHSP.

21. Conversione OSM -> SUMO

`convert_to_osm.py`, nonostante il nome, converte i file `.osm` in reti SUMO `.net.xml` usando `netconvert`:

```
netconvert --osm-files input.osm -o output.net.xml
```

Le opzioni aggiuntive:

- rimuovono geometrie non necessarie;
- uniscono junction vicine;
- provano a riconoscere i semafori.

I risultati vengono usati in `net_files`.

22. Ruolo di `sumo_visualize.py`

Questo script costruisce un secondo grafo leggendo `net.xml` di SUMO.

Per ogni edge conserva:

```
junction iniziale -> junction finale, ID edge, lunghezza
```

Poi applica Dijkstra, usando la lunghezza come costo, per ottenere la sequenza di edge SUMO piu corta tra START e GOAL.

Questo calcolo e distinto dal piano ENHSP:

- ENHSP ottimizza il problema PDDL e il tempo totale modellato;
- Dijkstra in `sumo_visualize.py` cerca il percorso SUMO di lunghezza minima.

In modalita dinamica lo script legge START e GOAL da un problema PDDL, li mappa sulle junction SUMO e calcola gli edge da visualizzare.

23. Differenza tra i due percorsi

Il progetto contiene due meccanismi di routing:

```
Webapp/PDDL: ENHSP -> minimizza total-time  
SUMO:        Dijkstra -> minimizza lunghezza degli edge
```

I percorsi possono differire perché usano grafi, conversioni e costi diversi. La simulazione SUMO non sta eseguendo direttamente il piano ENHSP: ricostruisce un percorso sulla propria rete tra gli stessi estremi.

24. Ruolo di `encoder/encoder.py`

L'encoder non calcola percorsi. Legge le way OSM e produce rapporti testuali contenenti:

- nome della strada;
- ID della way;
- tipo `highway`;
- velocità;
- senso unico;
- numero di nodi;
- numero di tratti con lo stesso nome.

Serve per ispezionare in modo leggibile il contenuto dei file OSM.

25. Ruolo di `build_problems.py`

Questo script genera automaticamente problemi predefiniti per le zone media e grande. Per ogni zona:

1. legge il file OSM; 2. costruisce il grafo contratto; 3. seleziona il sottografo; 4. sceglie automaticamente START e GOAL; 5. scrive `problem_media.pddl` o `problem_grande.pddl`.

La webapp usa logica simile, ma permette di caricare un file e cambiare gli estremi prima della risoluzione.

26. Ruolo di `pddl_files/run.py`

`run.py` cerca il file JAR di ENHSP, seleziona il problema della zona e avvia:

```
java -jar enhsp.jar -o domain.pddl -f problem_ZONA.pddl -s aibr
```

Poi stampa il piano quando ENHSP dichiara il problema risolto.

27. File principali e responsabilità

```
download_dublin_map.py  
Scarica le mappe OSM con OSMnx.
```

```
build_problems.py  
Costruisce il grafo, seleziona nodi, sceglie start/goal e genera PDDL.
```

```
webapp/app.py  
Backend completo: caricamento, grafo, controlli, PDDL, ENHSP e statistiche.
```

```
webapp/templates/index.html
  Interfaccia: mappa, selezione start/goal, chiamate API e risultato.

pddl_files/domain.pddl
  Definisce movimento, guida, arrivo e costi.

pddl_files/run.py
  Avvia ENHSP sui problemi salvati.

convert_to_osm.py
  Converte OSM in rete SUMO tramite netconvert.

sumo_visualize.py
  Calcola con Dijkstra un percorso sulla rete SUMO e lo visualizza.

encoder/encoder.py
  Produce report leggibili sulle strade OSM.
```

28. Esempio completo semplificato

Rete:

```
A --100 m--> B --200 m--> C
                \
                --150 m--> D
```

Il problema PDDL contiene:

```
(at A)
(road A B)
(road B C)
(road B D)
(:goal (at D))
```

Il planner produce:

```
(start-move A B)
(start-move B D)
```

La webapp ricostruisce:

```
A -> B -> D
```

e somma distanza, tempo e semafori dei due archi.

29. Punti importanti da ricordare

- Il file OSM descrive la rete, ma non sceglie START e GOAL.
- I nodi di una strada sono indicati dagli `<nd ref="...">` dentro la way.
- Il progetto contrae i nodi intermedi e conserva soprattutto gli incroci.
- Lo START automatico e il nodo con piu uscite.
- Il GOAL automatico e il nodo raggiungibile geograficamente piu lontano.
- La BFS controlla la raggiungibilita rispettando i sensi unici.
- L'utente puo cambiare START e GOAL dalla webapp.
- ENHSP calcola il percorso PDDL minimizzando il tempo modellato.
- `sumo_visualize.py` usa invece Dijkstra sulla rete SUMO.

- I due sistemi di routing sono collegati, ma non sono lo stesso algoritmo.

30. Comandi principali

Esempi da eseguire nella cartella **files**:

```
python download_dublin_map.py
python build_problems.py
python pddl_files\run.py media
python encoder\encoder.py
python webapp\app.py
python sumo_visualize.py piccola
```

Per la conversione SUMO deve essere disponibile il comando **netconvert**. Per il planner devono essere disponibili Java ed ENHSP.